

2023 牛客

暑期多校训练营第一场

懵哥



难度分布

预估难度分布:

- 签到: DJ
- Easy: HKL
- Mid: ABM
- Hard: CEG
- 防 AK: FI



A - Almost Correct

题意

构造一个排序网络，使其能恰好排序除一个给定 01 串外的所有 01 串。01 串长度为 n 。

数据范围： T 组测试，每组 $n \leq 16$ ， $T \leq 100$ 。



A - 题解

一个可行的思路：恰好针对给定的 01 串的某一对 01 让它们反序。

记 01 串 s 中 0 所在位置的下标集合为 S_0 ，1 所在位置的下标集合为 S_1 （下标从 1 开始）。令 S_0 中最大值为 r ， S_1 中最小值为 l （显然 $l \leq |S_0| < r$ ，因为 s 没被排好序）

A - 题解

做法

- 将 S_0 中最大值挪到最右边, S_1 中最小值挪到最左边, 即依次执行 $\forall x \in S_0, (x, r)$ 和 $\forall x \in S_1, (l, x)$ 。
- 对 $\{1, 2, 3, \dots, n\} - \{l, r\} = \{1, 2, \dots, l-1, l+1, \dots, r-1, r+1, \dots, n\}$ 正常排序, 即去掉下标 l, r 对其他元素排序。
- 依次执行
 $(r, |S_0| + 1), (r, |S_0| + 2), \dots, (r, r-1), (r, n), (r, n-1), \dots, (r, r+1),$
 $(l, |S_0|), (l, |S_0| - 1), \dots, (l, l+1), (l, 1), (l, 2), \dots, (l, l-1)。$

A - 验证

下面使用一个例子给出：给定串为 011001，另举 100010 和 010001。本例中 $S_0 = \{1, 4, 5\}$, $S_1 = \{2, 3, 6\}$, $l = 2, r = 5$ 。

- 第一步操作：011001, 100010, 001001。
- 第二步操作：010101, 000011, 000101。
- 第三步操作：001011, 000011, 000011。

前两步操作维持了 l, r 的反序，同时将其他修正。在第三步中，若两者皆错，不难发现 r 上的 0 至多只能移动到 01 分界线之后，而 l 上的 1 则移动到 01 分界线之前，最后一定是让指定串在 01 分界点上出错。

而如果 l, r 中有一个是正确的，则会让 01 分界线向前或向后推进，使得恰好排序。

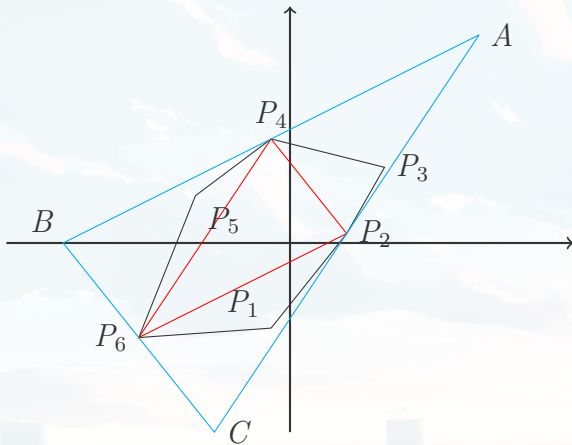
B - Anticomplementary Triangle

题意

平面上给定 n 个点 $\{P_n\}$ 构成一凸包 C ，找到三个点 P_r, P_s, P_t ，使得以这三点为中点三角形的三角形 $\triangle ABC$ 满足凸多边形 C 完全在其内部，如下图所示。

数据范围： $1 \leq n \leq 10^6$ 。点范围 $|x|, |y| \leq 10^9$ 。

B - Anticomplementary Triangle



B - 题解

结论

若 $\triangle P_r P_s P_t$ 面积为极大值（即三点中任意一点向相邻方向移动一个点均会使得三角形面积减小），则符合条件。

证明

不失一般性的，仅考虑 $P_r P_s$ 这条边与 P_t 的关系，且通过旋转凸包使得 $P_r P_s$ 处于水平。

由于面积是极大值，则 $P_{t-1} P_t$ 的斜率一定为负，而 $P_t P_{t+1}$ 一定为正。此处 P_{t-1} 表示 P_t 的前一个点， P_{t+1} 为后一个点。即， P_t 是整个凸包上 $P_r P_s$ 截出来的优弧上距离 $P_r P_s$ 直线最远的点。

B - 题解

证明

考虑如何根据重点构造中点三角形——过 P_t 做 P_rP_s 的平行线 ℓ 。由于 P_t 距离 P_rP_s 已经最远，则整个凸包一定在直线 ℓ 的一侧。其他两边同理，则构造出来的三条边均将整个凸包划在直线的一侧，因而这个三角形可以包括整个多边形。

B - 思路和做法

想到上述结论其实并不困难——考虑固定其中一条边 BC ，我们需要找到一个离这条边最远的一点 A 。因为是过该点 A 做 BC 的平行线 ℓ ，只有过最远的点才能让整个凸包都在 ℓ 的一侧。

做法相对简单：

- 固定一个点 $P_r = P_1$ 做初始点，随机再选择一个点 P_s ，移动 P_t 使得面积最大。
- 依次更新 P_r, P_s, P_t ，使得单它一个点移动的时候面积会增大。

这样做的复杂度为 $\mathcal{O}(n)$ 。假设 r 逆时针移动，则 r, s, t 三点均逆时针移动，并且 r 无法再回到 P_1 。



B - 思路 and 做法

注意，由于点的范围过大，达到 10^9 数量级，因而请使用整型而非浮点型进行计算。**计算几何能不用浮点数就一定不要用浮点数。**

对于本题而言，三角形面积可以通过叉积计算得到。

扩展：凸包

直观理解：想象每个点都是一个钉子，使用一个橡皮筋拉伸到无穷远处，遇到钉子就不会继续内缩。让其内缩到无法内缩为止，这样得到的图形就是这个点集的凸包。

凸包一个很重要的性质是每个内角均小于 180° 。可以使用 Andrew 算法在 $\mathcal{O}(n \log n)$ 的复杂度内求出一个点集的凸包。

C - Carrot Trees

题意

给一个长为 n 的序列，初始状态全为 0。有两种操作，第一种将区间 $[l, r]$ 内的数均增加 x ，第二种对区间 $[l, r]$ 内的每个数如果大于或等于 k 就减去 k (k 在所有操作中相等)。你需要执行 m 次操作，输出执行完所有操作后所有数一共被减了多少次。

数据范围： $1 \leq n \leq 10^6$, $1 \leq m \leq 2 \times 10^5$, $1 \leq x, k \leq 10^9$ 。

C - 题解

引入借位思想：使用 $a = bk + c$ 的形式来表示当前的高度。

- 操作一，直接令 $c_{i+1} = c_i + x$ 。
- 操作二，若 $a_i \geq k$ （即 $c_i + kb_i \geq k \Rightarrow c_i - k \geq -kb_i$ ），则令 $c_{i+1} = c_i - k$ ；否则令 $c_{i+1} = c_i - k$ 且 $b_{i+1} = b_i + 1$ 。

结论

引理：使用上述算法计算，所有操作后 $b_m = \left\lceil -\frac{\min_i c_i}{k} \right\rceil$ 。

C - 结论

证明

经过一次操作一， $c_{i+1} \geq c_i$ ，则 $\min_{1 \leq j \leq i} c_j = \min_{1 \leq j \leq i+1} c_j$ ，因而 $b_i = b_{i+1}$ ，与上述算法执行过程相同。考虑操作二的两种情况：

- 当 $a_i \geq k$ 时，有 $c_{i+1} = c_i - k \geq -kb_i$ ，因而 $b_{i+1} = b_i \geq -\frac{c_i}{k} + 1 \geq \lceil -\frac{c_i}{k} \rceil$ 。

显然 $c_{i+1} < c_i$ ，由数学归纳法可得若 $b_i = \left\lceil \frac{\max_{0 \leq j \leq i} (-c_j)}{k} \right\rceil$ 成立则

$b_{i+1} = \left\lceil \frac{\max_{1 \leq j \leq i+1} (-c_j)}{k} \right\rceil$ 成立，即 $b_{i+1} = \left\lceil -\frac{\min_j c_j}{k} \right\rceil$ 。

- 当 $a_i < k$ 时， $0 \leq c_i + kb_i < k$ 可得 $c_i \geq -kb_i$ 。由此时的操作 $b_{i+1} = b_i + 1$ 可得 $c_i \geq -k - kb_{i+1}$ ，则与第一种情况证明方法相同。



C - 做法

考虑离线处理。首先前缀和计算 c_i ，然后对于每种操作，前缀和它们的影响。即第 i 时刻对于区间 $[l, r]$ 的修改操作，转移到对 l 和 $r+1$ 的操作，表示当前点与之前点操作序列的不同。然后按时间顺序依次处理这些操作。这样线段树上时刻维护了当前正在处理的最小值信息。

时间复杂度 $\mathcal{O}(n \log m)$ 。



D - Chocolate

题意

$n \times m$ 的巧克力，Kelin 先手 Walk Alone 后手拿走一块矩形的巧克力，谁拿走 (n, m) 处的巧克力谁输，问结果。

数据范围： $1 \leq n, m \leq 10^9$ 。



D - 题解

结论

只有 1×1 的时候 Walk Alone 会赢，其余情况均是 Kelin 赢。

证明

1×1 显然是 Kelin 先手拿 $(1, 1)$ ，直接输。对于 $1 \times n$ 的时候，Kelin 先手拿走 $(1, n-1)$ ，只留一块给 Walk Alone，显然 Walk Alone 就输了。 $m \times 1$ 同理。

D - 题解

证明

考虑 $n \times m$ 的情况。Kelin 先手拿最左下角的一块，那么 Walk Alone 要么将巧克力吃成原来的 $n \times k$ 的形状，要么保持原有的矩形缺角的形状。对于后者，Kelin 可以在上面若干行接着吃，使得缺角形状保持，而且一定可以合法操作。也就是说，Walk Alone 拿到的一定是缺角形状，而 Kelin 要么面对的是 $n \times m$ 的完整矩阵形状，要么就一定可以操作一步，将现有局面改变到 Walk Alone 拿到的缺角形状。按照上面的归纳， $1 \times k$ Kelin 必胜，因而 $n \times m$ Kelin 必胜，而面对缺角形状的 Walk Alone 必败。

E - Heap

题意

题意：给定一个以 1 为根的有根树，点个数为 n ，每个点有点权 a_i 。现要求给每个点重新赋点权 b_i ，且要求 $b_{p_i} \leq b_i$ ，其中 p_i 为 i 的父亲。

输出最小的 $\sum_{i=1}^n (a_i - b_i)^2$ 。

$1 \leq n \leq 2 \times 10^5$, $0 \leq a_i \leq 10^9$ 。

E - 弱化版题目

本题前置知识：Slope Trick。首先先看本题的弱化版：

弱化版题目

给定一个以 1 为根的有根树，点个数为 n ，每个点有点权 a_i 。现要求给每个点重新赋点权 b_i ，且要求 $b_{p_i} \leq b_i$ ，其中 p_i 为 i 的父亲。

输出最小的 $\sum_{i=1}^n |a_i - b_i|$ 。

$1 \leq n \leq 2 \times 10^5$, $0 \leq a_i \leq 10^9$ 。

E - 弱化版题目做法

首先考虑一个最基础的 dp: $f_u(x)$ 表示 u 节点子树下 $b_u \geq x$ 的最小代价, $g_u(x)$ 是 $b_u = x$ 的最小代价, 有下式:

$$g_u(x) = |x - a_u| + \sum_{v \in \text{son}_u} f_v(x) \quad (1)$$

E - 弱化版题目做法

将 $f_u(x)$ 和 $g_u(x)$ 定义域范围扩展至全体实数，可以证明引理 1。

引理 1

$f_u(x)$ 是一个可导且单调递增的函数，且形如 $f_u(x) = c_u + \sum_{i \in S_u} r(x - i)$ ，其中：

- $r(x - a_u) = \text{ReLU}(x - a_u) = (x - a_u)[x \geq a_u]$ ，其中 $[x \geq a_u]$ 为艾弗森括号，表示当 $x \geq a_u$ 时值为 1，反之为 0。
- S_u 为 u 节点子树中全部 a_u 构成的可重集合。
- c_u 为一确定常数。

E - 引理 1 证明

递增性显然成立，下面使用数学归纳法证明该结论。

首先我们考虑叶节点 $g_u(x)$ 与 $f_u(x)$ 的形式—— $g_u(x) = |x - a_u|$ ， $f_u(x) = r(x - a_u) = (x - a_u)[x \geq a_u]$ ，因而 $f_u(x)$ 是一个单调递增且可导的函数，且符合归纳形式。

注：ReLU 函数在 $x = 0$ 处严格意义上不可导，但是为方便后文的表述，不妨令 $r'(0) = r'(0^+) = 1$ 。

对其余非叶节点， $g_u(x) = |x - a_u| + \sum_{v \in \text{son}_u} f_v(x)$ 。由可导函数相加还是可导函数， $g_u(x)$ 也同样可导，且 $c_u + \sum_{i \in S_u} r(x - i)$ 形式具有可加性。对其做后缀 min 操作，则 $|x - a_u| \rightarrow r(x - a_u)$ ，因而 $f_u(x)$ 同样符合原形式。归纳得证。

E - 弱化版题目做法

考虑将 $f_u(x) = c_u + \sum_{i \in S_u} r(x - i)$ 带入原转移式:

$$\begin{aligned} g_u(x) &= |x - a_u| + \sum_{v \in \text{son}_u} c_v + \sum_{v \in \text{son}_u} \sum_{i \in S_v} r(x - i) \\ &= (-x + a_u + 2r(x - a_u)) + \sum_{v \in \text{son}_u} c_v + \sum_{v \in \text{son}_u} \sum_{i \in S_v} r(x - i) \quad (2) \\ &= -x + a_u + \sum_{v \in \text{son}_u} c_v + \sum_{i \in S'_u} r(x - i) \end{aligned}$$

其中 $S'_u = \bigcup_{v \in \text{son}_u} S_v \cup \{a_u, a_u\}$ 。

E - 弱化版题目做法

不妨对 $g_u(x)$ 求导以研究极小值。

当导函数第一次大于等于 0 时, $x = \min S'_u$ 。令 $x_0 = \min S'_u$, 当 $x = x_0$ 时 $g_u(x)$ 取到最小值 $a_u - x_0 + \sum_{v \in \text{son}_u} c_v$, 因此令

$$S_u = S'_u - \{x_0\}$$

$$c_u = a_u - x_0 + \sum_{v \in \text{son}_u} c_v \quad (3)$$

E - 弱化版题目做法

■ 当 $x \geq x_0$ 时有

$$\begin{aligned} f_u(x) &= g_u(x) \\ &= -x + x_0 + c_u + \sum_{i \in S'_u} r(x - i) \\ &= r(x - x_0) - x + x_0 + c_u + \sum_{i \in S_u} r(x - i) \\ &= c_u + \sum_{i \in S_u} r(x - i) \end{aligned} \tag{4}$$

E - 弱化版题目做法

■ 当 $x < x_0$ 时, 由于 $f_u(x)$ 是 $g_u(x)$ 的后缀 min, 因而

$$f_u(x) \equiv c_u + \sum_{i \in S_u} r(x - i) = c_u \quad (5)$$



E - 弱化版题目做法

用优先队列维护 S_u 做启发式合并，可以在 $O(n \log^2 n)$ 内求出答案。

E - 做法

接下来我们回到原问题，即目标函数为平方代价。

用 $g_u(x)$ 表示保证以 u 为根的子树满足所有父节点权值小于等于子节点，并且使得 u 的权值**恰好**为 x 的最小代价； $f_u(x)$ 表示使得 u 的权值**至少**为 x 的最小代价，即 $f_u(x) = \min_{y \geq x} g_u(y)$ 。可以得到以下转移：

$$g_u(x) = (x - a_u)^2 + \sum_{v \in \text{son}_u} f_v(x) \quad (6)$$

E - 做法

用上面同样的方法，可以得到

$$f_u(x) = c_u + \sum_{i \in S_u} (x - i)^2 [x \geq i] \quad (7)$$

其中 S_u 仅为关于 u 的集合。

E - 做法

这时形式较为复杂，对其做后缀 $\min$ 操作难度较大。

但是我们可以研究它的导函数以研究它的极小值情况。对转移式两边求导有

$$g'_u(x) = 2(x - a_u) + \sum_{v \in \text{son}_u} f'_v(x) \quad (8)$$

其中 son_u 表示 u 的儿子集合。

E - 做法

根据上面同样的方法，显然 $f_u(x)$ 是一个连续、递增、下凸的分段函数，因此 $f'_u(x) \geq 0$ ，且由 $f_u(x) = \min_{y \geq x} g_u(y)$ 可得

$$f'_u(x) = \max\{g'_u(x), 0\} \quad (9)$$

E - 做法

用可重集合 $S_u \subseteq \mathbb{R} \times \mathbb{N}$ 维护 $f_u(x)$ 的所有拐点以及拐点两侧的斜率差。
下用同样的方法证明引理 2:

引理 2

$$f_u(x) = \sum_{(i, \delta) \in S_u} \delta \cdot r(x - i) \quad (10)$$

E - 引理 2 证明

同样引入 $r(x) = x \cdot [x > 0]$, 假设 u 的所有儿子节点 $f_v(x)$ 都能表示成以下形式:

$$f_v(x) = \sum_{(i,\delta) \in S_v} \delta \cdot r(x - i) \quad (11)$$

E - 引理 2 证明

代入 $g'_u(x)$ 的转移式可得

$$\begin{aligned} g'_u(x) &= 2(x - a_u) + \sum_{v \in \text{son}_u} \sum_{(i, \delta) \in S_v} \delta \cdot r(x - i) \\ &= -2a_u + 2x + \sum_{(i, \delta) \in S'_u} \delta \cdot r(x - i) \end{aligned} \tag{12}$$

其中 $S'_u = \bigcup_{v \in \text{son}_u} S_v$ 。显然 $g'_u(x)$ 单调递增，且存在唯一零点 x_0 。

E - 引理 2 证明

考虑由于求后缀 $\min$ 使得 x_0 处增加的拐点 $(x_0, g''_u(x_0^+))$, 令

$$S_u = \{(i, \delta) \in S'_u \mid i > x_0\} \cup (x_0, g''_u(x_0^+)) \quad (13)$$

则 $f'_u(x) = \sum_{(i, \delta) \in S_u} \delta \cdot r(x - i)$, 归纳假设成立。

AC.

E - 做法

记 $f_u(x)$ 与 $g_u(x)$ 的最小值为 C_u 。不难注意到，当子树中存在拐点 (i, δ) 有 $i \leq x_0$ 时，由于已经在 x_0 极其左侧取得最小值，因而 x_0 左侧的全部拐点都可以删去。

在寻找这样的 x_0 时，可以依次枚举 S'_u 中相邻的拐点，找到零点存在的区间然后求出 x_0 ，再删去左侧的拐点。

E - 做法

由于 $g(x)$ 的极小值在 x_0 处取到，代入 $g_u(x)$ 的转移式可得

$$\begin{aligned} C_u &= (x_0 - a_u)^2 + \sum_{v \in \text{son}_u} f_v(x_0) \\ &= (x_0 - a_u)^2 + \sum_{v \in \text{son}_u} C_v + \sum_{v \in \text{son}_u} \int_{-\infty}^{x_0} f_v(x) \, dx \end{aligned} \tag{14}$$

通过启发式合并可以在 $O(n \log^2 n)$ 的时间内求出所有集合 S_u 与每个函数对应的最小值 C_u 。总时间复杂度 $O(n \log^2 n)$ 。



F - 题意

题意

平面上有 n 个圆 $C_1, C_2, \dots, C_n$, 要求画一个圆 O (或直线) 经过尽可能多的圆。

数据范围: $1 \leq n \leq 150$, 坐标范围 $0 \leq |x|, |y| \leq 10^3$ 。



F - 解法

当 $n \leq 3$ 时答案为 n 。当 $n > 3$ 时，最优解一定能通过调整使得与至少两个圆外切。

转化题意

枚举 $i, j (i \neq j)$ ，假设圆 O 与 C_i, C_j 外切，求 O 最多能与多少 C_k 相交。

思路：看到此类圆和直线等价、圆圆相切的题，第一反应就应当是反演变换。

F - 解法

记 C_i, C_j 的半径分别为 r_i, r_j , 圆心分别为 o_i, o_j (用复数表示)。假设对复平面进行反演映射 $f(z) = 1/(z - o)$, 则反演后 C_i 的圆心和半径分别变为

$$\begin{aligned} o'_i &= \frac{1}{2} \left(\frac{1}{o_i - r_i e - o} + \frac{1}{o_i + r_i e - o} \right) \\ &= \frac{o_i - o}{(o_i - o)^2 - (r_i e)^2} \\ r'_i &= \frac{1}{2} \left| \frac{1}{o_i - r_i e - o} - \frac{1}{o_i + r_i e - o} \right| \\ &= \frac{r_i}{(o_i - o)^2 - (r_i e)^2} \end{aligned} \tag{15}$$

其中 $e = (o_i - o)/|o_i - o|$ 。类似地可以计算 C_j 的圆心和半径。

F - 解法

令反演后 C_i 与 C_j 圆心位置相同，可以得到

$$\frac{o_i - o}{(o_i - o)^2 - (r_i e)^2} = \frac{o_j - o}{(o_j - o)^2 - (r_j e)^2} \quad (16)$$

解得 $Ao^2 + Bo + C = 0$ ，其中

$$\begin{aligned} A &= o_i - o_j \\ B &= (r_i^2 - r_j^2) e^2 - (o_i^2 - o_j^2) \\ C &= (o_i r_j^2 - o_j r_i^2) e^2 + o_i o_j (o_i - o_j) \end{aligned} \quad (17)$$

这里需要注意的是，复平面上不同的点经过反演映射后必然映射到不同的点，但是反演前后圆心所在的点会发生变化。

F - 解法

取圆 C_i 内部的点 o ，则反演后除 C_i, C_j 外的所有圆都在 C_i 内部、 C_j 外部，且 O 与 C_i 内切，与 C_j 外切。

对于每个圆 C_k ，可以计算出 O 与 C_k 相交时 C_iO 的辐角取值区间，通过扫描线可以求出最多相交的圆的数量。

总复杂度 $O(n^3 \log n)$ 。

G - LCRS Transform

题意

给出一种对 n 个点的二叉树有根树（树根在 1）的操作：

- 由深到浅的遍历树上的每个节点 u ，将 u 的右儿子 r_u 变为 u 的左儿子 ℓ_u 的右儿子。
- 然后对于所有只有右儿子没有左儿子的节点，将其右儿子变为左儿子。

问进行 k 次操作后有多少棵树能变成给定的树。

数据范围： $1 \leq n \leq 10^4$ ， $0 \leq k \leq 100$ 。

G - 结论

结论 1

每次操作后树的前序遍历不变。

考虑按照前序遍历的顺序把点重新标号。这样标号后点 u 的左儿子 ℓ_u 满足 $\ell_u = u + 1$ ，且每次操作后边 (u, r_u) 会变为 $(\min\{u + 1, r_u - 1\}, r_u)$ ——前者是左儿子，后者是右儿子迁移到左儿子。

k 次操作后变为 $(\min\{u + k, r_u - 1\}, r_u)$ 。

G - 结论

考虑 k 次操作后的树最开始的样子。考虑两种边：

- 对于操作完成之后的树上的每条满足 $r_u > u + 1$ 的边 (u, r_u) ，最开始的树中一定存在边 $(u - k, r_u)$ ，因而需要保证此时的 $u > k$ 且这些边对应的“区间”不相交（这里定义两条边 (x, y) 与 (ℓ, r) 相交为 $\ell \leq x < r < y$ 或 $x < \ell < y \leq r$ ），否则答案为 0，因为这样就无法建立一条 $u \rightarrow r_u$ 的边。
- 对于满足 $\ell_u = u + 1$ 条件的边 (u, ℓ_u) ，最开始的边可以为 $(u - i, \ell_u)$ ($0 \leq i \leq k$)，表示这条边在第 i 步开始就从连接右儿子变成连接左儿子，然后相对关系不再变化。



G - 转化题意

转化题意

一个长度为 n 的线段中给定一些长度大于 k 的区间，要求在其中添加一些长度不超过 k 的区间，使得所有区间互不相交，问一共有多少种方案。

G - 做法

显然新添加的区间不可能跨过给定的区间，因此可以认为这些给定的区间把整个线段划分为了若干段，单独考虑每一段的贡献。令 f_i 表示长度为 i 的区间的方案数，其中 $f_0 = f_1 = 1$ 。

对于 $i \geq 2$ 有

$$f_i = f_{i-1} + \sum_{l=2}^{\min\{k+1, i\}} f_{i-l} \cdot f_{l-2} \quad (18)$$

G - 做法

令 $g = \sum_{i=0}^{k+1} f_i x^i$ 。当 $i \leq k+1$ 时 g 有生成函数等式：

$$g = xg + x^2 g^2 + 1 \quad (19)$$

解得

$$g = \frac{1 - x - \sqrt{1 - 2x - 3x^2}}{2x^2} \quad (20)$$

题外话：这里和 Z 变换的移序特性较为相似。

G - 做法

此时通过多项式求逆求出 f 数组的前 $k+1$ 项，即得到了递推的系数项。

考虑对于 $i > k+1$ 时递推式可化简为 $f_i = f_{i-1} + \sum_{l=2}^{k+1} f_{i-l} \cdot f_{l-2}$ ，转写成多项式形式为：

$$f = fx + fgx^2 + 1 \quad (21)$$

即 $f = \frac{1}{1 - x - gx^2}$ 。可以用多项式求逆和多项式开方在 $O(n \log n)$ 时间内求解。

H - Matches

题意

给定两个长度为 n 的序列 $\{a\}_{i=1}^n$ 和 $\{b\}_{i=1}^n$ ，现在可以选择其中一个序列交换其中的两个数字，问经过至多一次操作后最小的 $\sum_{i=1}^n |a_i - b_i|$ 。

$1 \leq n \leq 2 \times 10^5$, $0 \leq |a_i|, |b_i| \leq 10^{12}$ 。

H - 结论

定义

- 对于一对 (a_x, b_x) 和 (a_y, b_y) , 不妨设 $a_x < b_x$, 若 $b_y < a_y$ 则称他们成反序关系, 若 $a_y < b_y$ 则为正序。
- 若一对 (a_x, b_x) 与 (a_y, b_y) 在数轴上覆盖范围有重叠, 且存在独占一段的情况, 称为相交。
- 若一对 (a_x, b_x) 与 (a_y, b_y) 在数轴上覆盖范围有重叠, 一段完全包络另一段, 称为包络。
- 若一对 (a_x, b_x) 与 (a_y, b_y) 在数轴上覆盖范围没有重叠, 称为不交。



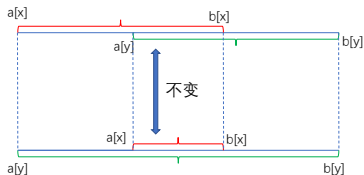
H - 结论

结论

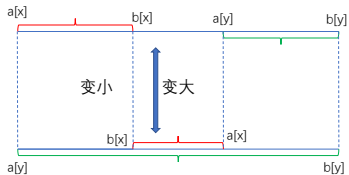
- 正序相交 = 正序包络
- 正序不交 < 反序包络
- 反序不交 < 反序相交

H - 结论

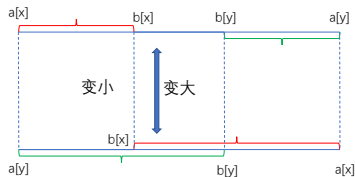
正序相交=正序包络



正序不交<反序包络



反序不交<反序相交



H - 解法

- 将所有的 (a_i, b_i) 分为 $a_i < b_i$ 与 $a_i > b_i$ 两类，记为 S 和 T 。同时计算初始值。
- 对于 S 中每个元素，在 T 中二分找到包络当前区间的区间，更新反序包络到正序不交的交流减少量。
- 对于 S 中每个元素，在 T 中二分找到相交范围最大的线段（最近的左端点，最近的右端点），更新反序相交到反序不交的交流减少量。



I - Random

题意

给定对 x 进行 m 次左移/右移并异或的函数 $\text{rand}(x)$ ，问期望对 $[0, 2^n - 1]$ 上均匀随机分布的 x 执行多少次 rand 可以变回 x 本身。

数据范围： $1 \leq n \leq 10^5$ ， $1 \leq \log_2 x \leq 32$ 。



I - 做法

核心做法

整体做法：对映射矩阵的极小多项式的阶枚举因数统计答案，进行莫比乌斯反演。



I - 做法

引理 1

这个随机数生成器是一个 k 维模 2 ($\mathbb{F}_2$) 的向量空间 V 中的线性映射 A , 且 A 可逆。

I - 做法

证明：不妨以 $a_i > 0$ 为例。该变换的矩阵表示为：

$$\begin{bmatrix} 1 & & \cdots & 1 \\ & 1 & \cdots & \\ & & \ddots & \\ & & & 1 \\ & & & & 1 \end{bmatrix} \quad (22)$$

可以通过列倍加实现从单位阵到上述矩阵的转化。而这一操作不改变矩阵的秩。所以该变换矩阵可逆。

I - 做法

引理 2

对于任意输入 x ，周期 t 必然存在，即 $A^t \mathbf{x} = \mathbf{x}$ ，即这个线性映射意义下的阶，下记为 $\text{ord}(\mathbf{x})$ 。

证明：如果把每个向量 $\mathbf{x}$ 看成一个节点，向量空间 V 看成一张图 G ，那么显然每个点都有一个后继 $A\mathbf{x}$ ，也都有一个前驱 $A^{-1}\mathbf{x}$ 。这就是说 G 一定是由若干个环构成的。于是我们证明了每个 $\mathbf{x}$ 的周期 $t_{\mathbf{x}}$ 必然存在，且定义为它的阶 $\text{ord}(\mathbf{x})$ 。

I - 做法

因而最后的答案为 $\frac{1}{2^k} \sum_{i=0}^{2^k-1} \text{ord}(i)$ ，其中 i 表示 i 的二进制表示对应的向量 x_i 。当然，直接统计的时间复杂度是不可接受的。

但是以 t 为周期（不一定是最简周期）的向量 x 的数量却可能是容易的。

I - 做法

引理 3

以 t 为周期（不一定最简）的向量数量为 $2^{k-\text{rank}(A^t-E)}$ 。

证明：符合条件的向量数量本质上是方程 $A^t \mathbf{x} = \mathbf{x}$ 的解集大小。也就是求 $|\ker(A^t - E)|$ 或者说 $2^{\text{nul}(A^t-E)}$ ，其中 $\ker$ 指的是零空间， nul 指的是零化度， E 是单位阵。根据秩-零化度定理，解个数就是 $2^{k-\text{rank}(A^t-E)}$ 。

I - 做法

考虑通过容斥求出每个向量具体的阶，以 s ($s|t$) 为阶的向量的答案可以通过对以 t 为周期的向量数量容斥求出。这是因为到每一个以 t 的因子为最小周期的 x 也会以 t 为周期，故而我们需要容斥掉 t 的因子的贡献。

设 $f(t) = t \times 2^{k - \text{rank}(A^t - E)} \times 2^{-k}$ ，阶为 s 的向量对答案的贡献为 $g(s)$ ，则由上知 $f(t) = \sum_{s|t} g(s)$ 。若知 $f(t)$ 的值，莫比乌斯反演即可线性求得 $g(s)$ 。问题可转化为求 $f(t)$ 。

I - 做法

然而 s, t 的量级是难以确定的。上述操作仍不足以解决此题，甚至不一定会使得时间更优。我们注意到不少 s 贡献均为 0，因此尝试寻找 s 的性质和量级。有引理 4：

引理 4

所有 s 均是方程 $A^r = E$ 的解 r 的因子。

证明：由于 A 本质有限且可逆，不难发现 $\{A^r\}$ 构成一个循环群。根据群论的拉格朗日定理，有限群的循环子群的阶是其母群的阶的因数。

此处母群即是整个群 $\langle \{A^0 = E, A^1, \dots, A^{r-1}\}, \times \rangle$ ，而子群表示由阶为 s 的元素构成的子群 $\langle \{A^0, A^s, A^{2s}, \dots, A^{r-s}\}, \cdot \rangle$ ，显然有 $s \mid r - s$ 。

I - 做法

利用引理 4，我们知道最小周期 s 是循环子群的阶。在这里，我们可以认为其母群的阶即是最小的 r 使得 $A^r = E$ 。

引理 5

母群的阶 r 是方程 $X^r \equiv E \pmod{p(X)}$ 的最小正整数解。其中 $p(X)$ 是极小多项式，即一个 $\mathbb{F}_2$ 上的最低次首一多项式

$$p(X) \in \left\{ \sum_{i=0}^l p_i X^i \mid p_i \in \mathbb{F}_2, 1 \leq l < r \right\} \text{ 满足 } p(A) = 0。$$

I - 做法

证明：由于 A 满秩，因而 $\forall k, A^k \neq 0$ ，因而 $p(X) \nmid A^r$ 。所以不难注意到在模 $p(x)$ 的矩阵乘法运算作用下的代数系统 $\langle \{A^0, A^s, A^{2s}, \dots, A^{r-s}\}, \cdot \pmod{p(X)} \rangle$ 也是一个群。

由于 $A^r = E$ ，考虑对 $A^r - E = 0$ 进行因式分解，则 $p(A)$ 为 $A^r - E$ 中的一项或若干项的乘积。假设 r 是极小多项式 $p(X)$ 的阶，即最小的正整数满足 $\forall X, p^r(X) = E$ 的 r ，则必然有 $X^r - E = 0$ ，则可推出 $X^r \equiv E \pmod{p(X)}$ 。

而显然如果母群的阶 $r' < r$ ，则 $p^{r'}(X) \neq E$ ，则 $X^{r'} \neq E$ 矛盾。因而该方程的解就是原问题的解。



I - 做法

最终做法

方程 $X^r \equiv 1 \pmod{p(X)}$ 正是标准的离散对数问题，可以使用 BSGS 求解出 r ，然后对 r 分解出 t 计算答案，最后莫比乌斯反演即可解决。

I - 复杂度证明

如果能证明 r 的范围，那么则可以证明这样做的复杂度正确。

引理 6

$$r < 2^k$$

证明：对于 A 的特征多项式 $q(\lambda) = \prod_{i=1}^k q_i^{\alpha_i}(\lambda)$ ，由卡莱-哈密顿定理可得 $q(A) = 0$ 。因而利用引理 5 的结论，方程 $A^x \equiv E \pmod{q(X)}$ 的解一定是 r 的倍数，可以通过它框定 r 的范围。

I - 复杂度证明

考虑 $q(\lambda)$ 的结构，考虑进行一个类似于 `excr` 的过程——将 $q(\lambda)$ 根据每个不可约多项式进行分解，首先研究每个不可约质多项式下同余多项式的解范围，再升次，最后合并这些解。

它在线性空间中的意义是，每一个因式 $q_i^{\alpha_i}(\lambda)$ 都对应一个不变子空间，也就是 $q_i^{\alpha_i}(A)$ 的零空间，记作 $\ker(q_i^{\alpha_i}(A))$ 。可以验证，所有 $\ker(q_i^{\alpha_i}(A))$ 的直和就是 k 维全空间，即它们的基向量彼此线性无关。

I - 复杂度证明

现在我们考察每一个 $\ker(q_i^{\alpha_i}(A))$ 。对于全空间中的某一个向量 x ，它的周期即是它在每一个子空间里的分量的周期的 lcm。因此，我们只需要考察每一个子空间里的分量的周期的量级，求取 lcm 即可得出全空间里的每一个分量的周期的量级。

对于 $q_i^{\alpha_i}(X)$ ，我们考察它的不可约因子 $q_i(X)$ 。因为 $\mathbb{F}_2$ 上 d 次不可约多项式 $q(X)$ 是 $X^{2^d} - X$ 的因子，故而不可约多项式 q_i 的周期 $r_{i,1}$ 不会超过 2^d 。在这里，周期指的是最小正整数 $r_{i,1}$ 使得 $X^{r_{i,1}} \equiv E \pmod{q_i(X)}$ 。

I - 复杂度证明

为了籍此推出 $q_i^{k_i}(X)$ 的周期的量级，我们需要证明引理 6.1:

引理 6.1

若 $\mathbb{F}_2$ 上的不可约多项式 $p(X)$ 的周期是 d ，那么 $p^k(X)$ 的周期的上界（可以证明是紧上界）是 $d \times \text{bitceil}(k)$ 。

证明：因为 $p(X)|(X^d - 1)$ ，故而 $p^2(X)|(X^d - 1)^2$ 。在 $\mathbb{F}_2$ 上， $(X^d - 1) = (X^d + 1)$ ，于是 $p^2(z)|(X^d + 1)(X^d - 1) = X^{2d} - 1$ 。故而 $p^{\text{bitceil}(k)}(X)$ 的周期是 $d \times \text{bitceil}(k)$ 。又显然若 $i < j$ 则 $p^i(z)$ 的周期不大于 $p^j(X)$ 的周期，故而 $p^k(X)$ 的周期的上界是 $d \times \text{bitceil}(k)$ 。

I - 复杂度证明

由此我们得到了 r 的规模为 $\text{lcm}(2^{d_i} \times \text{bitceil}(k_i))$, 在这里, d_i 是不可约多项式 $p_i(X)$ 的次数。这个值是显著小于 $O(2^k)$ 的。

I - 复杂度证明

最后，考虑复杂度：

复杂度分析

这样做的复杂度的级别是 $O\left(nk^2 + k^3 + k\sqrt{2^k} + \max_{1 \leq i \leq 2^k} d^2(i)\right)$ 。

I - 复杂度证明

证明：梳理一遍整个算法流程：

- 首先我们预处理出映射矩阵 A ，这需要 n 次 $\mathbb{F}_2$ 上的矩阵乘法，复杂度是 $O(nk^2)$ 。

- 我们首先求 A 的极小多项式 $p(X)$ ：我们找到最小的 m 使得

$$\exists \{a\}_m \in \{\mathbb{F}_2\}_m, \sum_{i=0}^{m-1} a_i A^i = A^m, \text{ 即 } p(X) = X^m - \sum_{i=0}^{m-1} a_i X^i. \text{ 根据凯}$$

莱-哈密顿定理， A 一定是其特征多项式 $q(\lambda)$ 的根即 $q(A) = 0$ ，而特征多项式为 k 次，因而 $m \leq k$ 。故而这个部分的复杂度是 $O(k^3)$ 。

I - 复杂度证明

- 接下来，我们使用“BSGS”求出阶 r ， r 的规模在前文已经证明为 $O(2^k)$ ，故而这里需要 $O(k\sqrt{2^k})$ 的复杂度。
- 然后，我们以 $O(\sqrt{r}) = O(\sqrt{2^k})$ 的复杂度分解 r ，并枚举它的因数。考虑到 $d(n)$ 的规模足够小，可以不使用莫比乌斯反演， $d^2(n)$ 的暴力容斥也是可以接受的。

J - Roulette

题意

Walk Alone 初始有 n 块钱，如果每次投 x 元，有一半的概率输掉这 x 元，另一半概率赢得 $2x$ 元。现在 Walk Alone 采取下述策略投注：

- 如果上一把赢了，这一把投 $x_i = 1$ 元
- 如果上一把输了，这一把投 $x_i = 2x_{i-1}$ 元。

问 Walk Alone 有多大概率拿到 $n + m$ 元离开。

$1 \leq n, m \leq 10^9$ 。

J - 做法

结论

以“输输输……输赢”为一个周期，可以赢到一块钱。因而需要经过 m 个这样的周期。

考虑每一轮能赢钱的概率，即是不能连输到本金输光为止——如果现在有 x 元，找到最大的 r 满足 $2^r - 1 \leq x$ ，则概率为 $1 - \left(\frac{1}{2}\right)^r$ 。



J - 做法

考虑枚举 r ，对于每个 r ，该段 $([2^r - 1, 2^{r+1} - 2])$ 的 x 获胜的概率均相等，同时这样的 r 只有 $O(\log(n + m))$ 段。

配合快速幂可以在 $O(\log^2 n)$ 的复杂度内解决本题。

K - Subdivision

题意

给定一个无向图 $G(n, m)$ ，可以将其中任意一条边分裂成一条长度为任意的链（向边中插任意多个点），可以操作任意多次（也可以不操作）。问经过这样处理之后，从 1 号节点出发，至多走 k 步最多可以到多少个节点。

数据范围： $1 \leq n \leq 10^5$ ， $1 \leq m \leq 2 \times 10^5$ ， $1 \leq k \leq 10^9$ 。



K - 题解

从根节点走 k 步能到的点可以用 bfs 树刻画。构造完 bfs 树，考虑分裂的边在哪里。

结论

对于从 1 号节点开始的路径 $P_n = \{1, v_1, v_2, \dots, v_n\}$ ，如果需要分裂边以增加点数，那么在增加点数相同的情况下，分裂 (v_{n-1}, v_n) 最优。



K - 题解

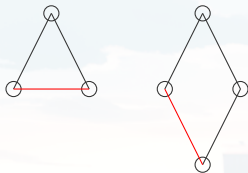
证明

不失一般性的，考虑仅增加一个点。假设在 v_i 处有 k_i 条不在 bfs 树上的边，其中 $i \in [1, n-1]$ 。如果这时修改 (v_j, v_{j+1}) ，则会让 $i \in [j+1, n-1]$ 所有的点连接的所有非 bfs 树边所能到达的点数减少一。那么显然当 $j = n-1$ 时这个损失的量最小。

K - 题解

考虑两种边：

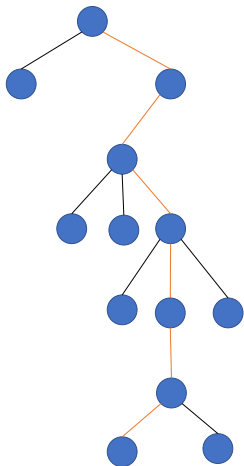
- 该边不在 bfs 树上。这种情况存在两个子类——该边连接了同层的两点、该边连接了不同层的两点，但是该边去除不影响图深度的计算，如下图中红色边所示。则这种情况下该边都可以分裂为无穷多个点，并且不影响其他点是否可以到达。



K - 题解

- 该边在 bfs 树上。考察对于 bfs 树的叶节点 u 及根节点到它的路径 $P_n = \{1, v_1, v_2, \dots, v_{n-1}, u\}$ ，首先我们对该路径上所有边不做处理，分两种小情况：
 - 走不到 u ，那么这条路径上不用分裂——因为显然分裂一次创造出来的新点不过是置换了路径中更深的一个点，并且由引理可知这样还会损失非树边的答案。
 - 走的到 u ，那么需要考虑 u 连接的非树边条数。如果 u 没有连接非树边，那么通过分裂 (v_{n-1}, u) 让 u 变为最深的节点可以让答案增加；否则则不要执行边分裂操作，因为这样增加了 (v_{n-1}, u) 路径上的答案会损失 u 连接的非树边的答案，哪怕只有一条边也不过是打平手。

K - 题解



左图中橙色线表示 bfs 树边，黑色线表示非树边。
不难发现，分裂深度越浅，影响到的非树边越多，对答案损失越大。

K - 做法

因而建立 bfs 树，模拟该过程即可，注意没有边的情况。时间复杂度 $\mathcal{O}(m)$ 。
注意不存在边的情况。



L - Three Permutations

题意

给定三个长度为 n 的排列 a, b, c , (x, y, z) 最开始为 $(1, 1, 1)$, 每过一秒变为 (a_y, b_z, c_x) 。 q 次询问求变成 (x', y', z') 的最短时间。

数据范围: $1 \leq n \leq 10^5$, $1 \leq q \leq 10^5$ 。

L - 做法

注意到每过 3 秒 (x, y, z) 变为 $((a \circ b \circ c)_x, (b \circ c \circ a)_y, (c \circ a \circ b)_z)$, 其中 $a \circ b$ 表示置换的复合运算。

尝试研究这三个复合置换。以 $f = a \circ b \circ c$ 为例, 三秒后 $x \rightarrow f_x$ 。因而可以考虑线性时间内计算除 f 置换上 1 经过多少轮迭代 (记为 α) 可以变成 x' , 以及多少轮迭代 (记为 β) 会回到 1, 其他两个置换同理。

枚举初始开始的值——从 0, 1, 2 秒开始计算, 问题转化成求最小的数字满足三个同余等式 $x \equiv \alpha \pmod{\beta}$ 。经典的 exCRT 问题, 时间复杂度 $O(q \log n)$ 。

扩展：EXCRT（中国剩余定理）

EXCRT 解决了下面的问题：

$$\begin{cases} x \equiv \alpha_1 \pmod{\beta_1} \\ x \equiv \alpha_2 \pmod{\beta_2} \\ \vdots \\ x \equiv \alpha_n \pmod{\beta_n} \end{cases} \quad (23)$$

首先将 β_i 质因子分解，然后合并这所有同质因子的方程，考察它们有没有冲突——即对于 $x \equiv \alpha_1 \pmod{p^{k_1}}$ 与 $x \equiv \alpha_2 \pmod{p^{k_2}}$ ，其中 $k_1 \geq k_2$ ，要求有 $\alpha_1 \equiv \alpha_2 \pmod{p^{k_2}}$ 。

扩展：EXCRT（中国剩余定理）

对于互质的 $\beta_1, \beta_2, \dots, \beta_n$ ，解形式为：

$$x \equiv \sum_{i=1}^n \alpha_i \cdot \frac{M}{\beta_i} \cdot \left(\left(\frac{M}{\beta_i} \right)^{-1} \bmod \beta_i \right) \pmod{M} \quad (24)$$

其中 $M = \prod_{i=1}^n \beta_i$ 。注意模逆元的对象。

M - Water

题意

给两个容积分别为 A, B 的水杯，每次可以执行以下的四种操作之一：

- 把其中一个水杯装满水。
- 把其中一个水杯中的全部水倒掉。
- 把其中一个水杯中现有的水全部喝完。
- 把一个杯子中的水尽可能转移到另一个水杯中，水不溢出。

为让懵哥喝掉恰好 x 体积的水，问最少要操作几次。

T 组询问， $1 \leq T \leq 10^5$ ， $1 \leq A, B \leq 10^9$ 。



M - 做法

记 $(r, s) = rA + sB$, 其中 $r, s \in \mathbb{Z}$ 。

显然两个杯子和喝掉的水在任意时刻都能表示成 (r, s) 的形式, 因此由裴蜀定理当 $\gcd(A, B) \nmid x$ 时答案为 -1 。

M - 解法

当 A, B 中分别装有 $(r_A, s_A), (r_B, s_B)$ 水时, 可以进行的操作如下:

- 装水: 将 A 变为 $(1, 0)$ 或将 B 变为 $(0, 1)$
- 倒水: 将 A 或 B 变为 $(0, 0)$
- 喝水: 将 A 或 B 变为 $(0, 0)$ 且喝掉的水 $+(r_A, s_A)$ 或 $+(r_B, s_B)$
- 将 A 转移到 B (B 转移到 A 同理):
 - 若 $(r_A + r_B, s_A + s_B - 1) < 0$, 则 A 变为 $(0, 0)$ 且 B 变为 $(r_A + r_B, s_A + s_B)$
 - 否则 A 变为 $(r_A + r_B, s_A + s_B - 1)$ 且 B 变为 $(0, 1)$

M - 操作次数上限

操作次数上限

假设 x 可以表示成 (r_0, s_0) ，至多需要 $\max\{2(r_0 + s_0), 2|r_0 - s_0| - 1\}$ 次操作。

- 当 $r_0 \geq 0, s_0 \geq 0$ 时， $\text{ans} \geq 2(r_0 + s_0)$ ：重复进行 r_0 次给 A 倒水、从 A 喝水，以及 s_0 次给 B 倒水、从 B 喝水即可。
- 当 $r_0 s_0 < 0$ 时， $\text{ans} \leq 2|r_0 - s_0| - 1$ 。不妨设 $r_0 > 0, s_0 < 0$ 。进行以下操作：

M - 操作次数上限

```
1:  $cnt_B \leftarrow 0$ ;  
2: for  $iter = 1, 2, \dots, r_0$  do  
3:   装满 A (操作 1) ;  
4:   while  $B \neq (0, 1)$  且  $A \neq (0, 0)$  do  
5:     A 转移到 B (操作 4) ;  
6:     if  $B = (0, 1)$  且  $cnt_B < |s_0| - 1$  then  
7:       倒空 B (操作 2) ;  
8:        $cnt_B \leftarrow cnt_B + 1$ ;  
9:   if  $A \neq (0, 0)$  then  
10:  喝 A (操作 3) ;
```


M - 操作次数下限

操作次数下限

$$\text{ans} \geq 2(r_0 + s_0)$$

假设某次操作前已经喝了 (r_M, s_M) 水，此时两个杯子中有水量 $(r_A, s_A), (r_B, s_B)$ ，记 $m = r_M + s_M$ ， $a = r_A + s_A$ ， $b = r_B + s_B$ ，构造函数 $f(m, a, b) = 2m + \max\{2a - 1, 0\} + \max\{2b - 1, 0\}$ ，注意该函数仅需要满足操作一次后函数值增大 1，且最后可以变化到 $2m$ ，这里仅给出其中一个符合条件的示例函数。

M - 操作次数下限

可以证明在当前局面下进行任意操作后 $f(m', a', b') \leq f(m, a, b) + 1$ 。由于初始状态为 $f(0, 0, 0) = 0$ ，目标状态 $f(r_0 + s_0, a, b)_{\min} = 2(r_0 + s_0)$ ，因此操作次数不少于 $2(r_0 + s_0)$ 。

M - 操作次数下限

操作次数下限

$$\text{ans} \geq 2|r_0 - s_0| - 1$$

当 $r_0 - s_0 \geq 0$ 时，记 $m = r_M - s_M$, $a = r_A - s_A$, $b = r_B - s_B$ 。

构造函数 $f(m, a, b) = 2m + \max\{2a - 1, 0\} + \max\{2b, -1\}$ 。注意该函数仅需要满足操作一次后函数值增大 1，且最后可以变化到 $2m - 1$ ，这里仅给出其中一个符合条件的示例函数。

M - 操作次数下限

可以证明进行任意操作后 $f(m', a', b') \leq f(m, a, b) + 1$ 。由于初始状态为 $f(0, 0, 0) = 0$ ，目标状态 $f(r_0 + s_0, a, b)_{\min} = 2(r_0 - s_0) - 1$ ，因此操作次数不少于 $2(r_0 - s_0) - 1$ 。

当 $r_0 - s_0 < 0$ 时同理可证操作次数不少于 $2(s_0 - r_0) - 1$ 。

M - 做法

结论

$$\text{ans} = \max\{2(r_0 + s_0), 2|r_0 - s_0| - 1\}$$

用 `exgcd` 求出离原点较近的 (r_0, s_0) ，并对附近的 $(r_0 + kB, s_0 - kA)$ 取 `min` 即可。

每组数据复杂度 $O(\log x)$ 。

THANKS!

AC.NOWCODER.COM